Builder

Build immutable HTTP requests with many optionals

Season 1 · GoF · Beginner

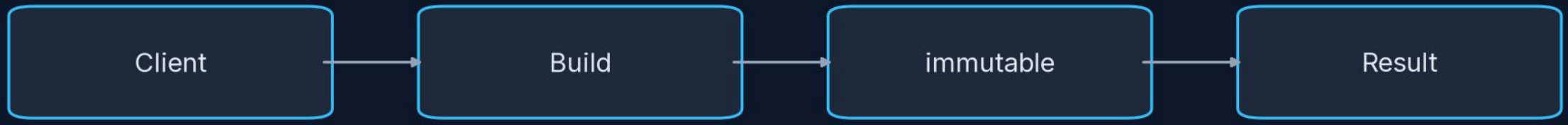
Today's Hook

- Problem: Build immutable HTTP requests with many optionals
- Naive pain cue: caught=url required
- We will refactor to Builder with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: Build immutable HTTP requests with many optionals
- Constraints: solve «Build immutable HTTP requests with many optionals» with Builder — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Build immutable HTTP requests with many optionals
- Success looks like: POST /x body={} auth=true

Structure



Builder: Build immutable HTTP requests with many optionals

Real-World Scenario

- Running example: Build immutable HTTP requests with many optionals
- Key types: HttpRequest, Builder
- Happy-path stdout cue: POST /x body={} auth=true
- Failure-path stdout cue: caught=url required

Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.builder.problem;

import java.util.*;

public final class NaiveHttp {
  public static String build(String url, String method, String body, boolean auth) {
    if (url == null || url.isBlank()) throw new IllegalArgumentException("url");
    return method + " " + url + " body=" + body + " auth=" + auth;
  }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Build immutable HTTP requests with many optionals
- Without Builder, call sites absorb the wrong details.
- Failure mode to show on camera: caught=url required
- New requirements force shotgun edits across the naive code.

Solution

```java
package com.buildwithwaqar.patterns.gof.creational.builder.problem;

import java.util.*;

final class HttpRequest {
  private final String url, method, body; private final boolean auth;
  private HttpRequest(Builder b){ url=b.url; method=b.method; body=b.body; auth=b.auth; }
  public String summarize(){ return method+" "+url+" body="+body+" auth="+auth; }
  public static final class Builder {
    private String url, method="GET", body=""; private boolean auth;
    public Builder url(String u){ this.url=u; return this; }
    public Builder method(String m){ this.method=m; return this; }
    public Builder body(String b){ this.body=b; return this; }
    public Builder auth(boolean a){ this.auth=a; return this; }
    public HttpRequest build(){
      if (url==null || url.isBlank()) throw new IllegalArgumentException("url required");
      return new HttpRequest(this);
    }
  }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.creational.builder.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Builder: immutable HTTP request ===");
    System.out.println("--- naive ---");
System.out.println(NaiveHttp.build("/x","POST","{}",true));
System.out.println("--- builder ---");
System.out.println(new HttpRequest.Builder().url("/v1/orders").method("POST").body("{}").auth(true).build().summarize());
try { new HttpRequest.Builder().method("GET").build(); } catch (IllegalArgumentException e) { System.out.println("caught="+e.getMessage()); }
  }
}
```

Walkthrough Output

```
=== Builder: immutable HTTP request ===
--- naive ---
POST /x body={} auth=true
--- builder ---
POST /v1/orders body={} auth=true
caught=url required
```

Step-by-Step Trace

- Match stdout to stages — look for: POST /x body={} auth=true
- Then the injected failure: caught=url required
- Re-run: java -cp ... 06-builder ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
// Java 25 direction for Builder
// Prefer records + sealed where they shrink the demo.
// Problem: Build immutable HTTP requests with many optionals
```

Trade-offs

- Use Builder when «Build immutable HTTP requests with many optionals» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Builder in product language.

Related Patterns

- Name the pattern usually confused with Builder and separate the forces.
- Sketch the roles for «Build immutable HTTP requests with many optionals» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Build immutable HTTP requests with many optionals
- Naive pain → Builder boundary → Demo stdout.
- Remember the failure cue: caught=url required
- Break a ProblemTest assertion, watch it fail, restore it.